Practical Short Tasks for Viva Preparation

1. Class aur Static Variables

Task:
Ek Library class banao jo books ka record rakhti hai. Isme ek class variable total_books ho jo saari libraries ke liye shared ho. Har library ka instance apna name aur book_list rakhe. Ek method add_book banao jo book add kare aur total_books ko update kare. Ek method show_books banao jo library ke books aur total books dikhaye.
Test: 2 libraries banao, dono mein books add karo, aur check karo ke total_books sahi update hota hai ya nahi.
Focus: Class vs instance variables, accessing/modifying class variables.

---

2. Composition aur Aggregation

Task:
Ek House class banao jo rooms ko store kare. Room class alag se banao jisme room_type (e.g., "Bedroom") aur area ho. House mein rooms list ke zariye aggregation use karo. Ek method add_room banao jo naye rooms add kare, aur total_area method jo saare rooms ka area calculate kare.
Test: Ek house banao, 3 rooms add karo, aur total area print karo.
Focus: Aggregation, has-a relationship.

---

3. Method Resolution Order (MRO)

Task:
Teen classes banao: Parent, Child1, aur Child2. Parent mein ek introduce method ho jo "I am Parent" return kare. Child1 aur Child2 is method ko override karen aur apna message return karen. Ek GrandChild class banao jo Child1 aur Child2 dono se inherit kare. introduce call karo aur MRO print karo.
Test: GrandChild ka object banao, introduce call karo, aur mro() method se order check karo.
Focus: Multiple inheritance, MRO understanding.

---

4. Decorators in Classes

Task:
Ek Employee class banao jisme salary attribute ho. @property use karke salary ka getter banao. @salary.setter se salary update karne ka method banao jo check kare ke salary positive hai. Ek computed property bonus banao jo salary ka 10% return kare.
Test: Employee banao, salary set karo, bonus check karo, aur negative salary set karne ki koshish karo.
Focus: Property decorators, getter/setter, computed properties.

---

5. Callable Objects

Task:
Ek Counter class banao jo call hone par ek internal counter ko increment kare aur current value return kare. __call__ method implement karo. Ek method reset bhi banao jo counter ko zero kare.
Test: Counter object banao, 3 baar call karo, value print karo, phir reset karo aur ek baar call karo.
Focus: Callable objects, __call__ method.

---

6. Modules aur Packages in OOP

Task:
Ek package school banao jisme do modules hon: students.py aur teachers.py. students.py mein Student class ho jisme name aur grade ho, aur ek display method jo details dikhaye. teachers.py mein Teacher class ho jisme name aur subject ho, aur ek display method. main.py mein dono classes import karo aur instances banao.
Test: Ek student aur ek teacher banao, aur unke details print karo.
Focus: Module organization, importing classes.

---

7. Advanced OOP Concepts (Singleton)

Task:
Ek Config class banao jo Singleton pattern follow kare. Isme ek dictionary settings ho jisme key-value pairs store hon (e.g., {"theme": "dark"}). Ek method get_setting banao jo key se value return kare.
Test: Do instances banao, ek mein setting add karo, aur check karo ke dono instances same dictionary share karte hain.
Focus: Singleton pattern, single instance.

---

8. Error Handling in OOP

Task:
Ek VendingMachine class banao jisme stock aur price attributes hon. Ek custom exception OutOfStockError banao. Ek method buy_item banao jo stock check kare aur agar stock khatam ho to exception raise kare.
Test: Machine mein 2 items rakho, 3 items kharidne ki koshish karo, aur exception handle karo.
Focus: Custom exceptions, try-except.

---

9. Testing OOP Code

Task:
Ek MathUtils class banao jisme square aur cube methods hon jo number ka square aur cube return karen. unittest use karke tests likho jo check karen ke square(3) = 9 aur cube(2) = 8 hai. Ek test case bhi likho jo negative number ke liye kaam kare.
Test: Tests run karo aur ensure karo ke sab pass hote hain.
Focus: Unit testing, assertions.

---

10. Best Practices in OOP (SOLID - Single Responsibility)

Task:
Ek Invoice class banao jo items store kare aur total calculate kare. Ise do alag classes mein divide karo: InvoiceCalculator jo total calculate kare, aur InvoicePrinter jo details print kare.
Test: Ek invoice banao, total calculate karo, aur details print karo.
Focus: Single Responsibility Principle, separating concerns.

---

11. Iterable

Task:
Ek NumberRange class banao jo ek range of numbers (start se end tak) iterable ho. __iter__ aur __next__ implement karo taake for loop mein numbers print hon.
Test: 1 se 5 tak ka range banao aur loop se print karo.
Focus: Iterables, __iter__ aur __next__.

---

12. Object-Based vs Object-Oriented Language

Task:
Ek Person class banao jisme name aur age attributes hon, aur ek introduce method jo details return kare. Inheritance use karke Student class banao jo Person se inherit kare aur grade attribute add kare.
Test: Ek Person aur ek Student banao, aur dono ke introduce method call karo.
Focus: Inheritance, OOP principles.

Tips for Viva Preparation

1. **Code Practice:** Har task ko Python mein code karo. Pehle pseudocode likho, phir implement karo. Errors ko debug karna seekho.
2. **Concepts Samjho:** Har task ke focus area ko samjho aur Viva mein explain karne ke liye 2-3 sentences tayaar rakho.
3. **Output Predict Karo:** Har task ka expected output pehle socho, phir code run karke verify karo.
4. **Viva Questions:** Har topic ke liye basic questions tayaar karo, jaise:
    - Class variable aur instance variable mein kya farq hai?
    - Composition aur aggregation mein difference kya hai?
    - @property ka use kyun hota hai?

Kaise Practice Karein

- Ek task ko 15-20 minutes mein solve karne ki koshish karo.
- Code ko comments ke saath likho taake logic clear rahe.
- Har task ke baad output check karo aur agar error aye to uska reason samjho.
- Viva ke liye har task ka concept verbally explain karne- Ek chhota sa note banao.